NEXUS-1
ROOT-CAUSE ANALYSIS ENGINE

Mechanism & Decision Architecture

How the Root Cause Graph reaches its conclusion — the deterministic causal engine, the MS SQL Server grounding database, and the locally-served DSLM explanation layer (Ollama, air-gapped, retrieval-grounded) with its anti-hallucination controls.

AUTHOR	Grigorios Agathangelidis — Electrical & Software Engineer
BUILD	NX1-094C79E0A366D34F
DATE	June 2026 · mechanism document
RELATES TO	Companion to the Root-Cause Analysis Engine roadmap (R0–R8 staged plan: it prices the work; this document specifies the mechanism) and to the Root Cause Graph — Analytical Methodology note (the three-incident rationale). Stack aligned to Project Roadmap v2.

HONEST SCOPE — READ FIRST

This document describes two clearly separated things. Part A is the mechanism the Phase-0 console runs today: a deterministic, engineered causal engine — every number it shows is computed by the working code. Part B is the production target: the inference core, the grounding database and the DSLM explanation layer. The DSLM is not running in this build; it is specified here as the architecture the roadmap prices. In both parts the system is advisory only — it informs operators and investigators and never feeds control or safety actuation.

1 · The decision principle: the engine decides, the model explains

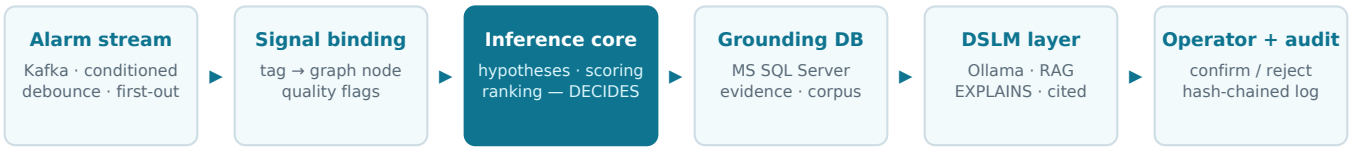
A root-cause conclusion in NEXUS-1 is never produced by a language model. It is produced by a deterministic, auditable inference core operating on an engineered causal graph — the same mathematics every time, reproducible from logged inputs. The locally-served language model (DSLM) sits strictly downstream of the decision: it retrieves the supporting evidence, composes the explanation with citations, and answers operator questions about a conclusion that has already been reached. It cannot create a hypothesis, change a score, or reorder the ranking.

This split is the document's central design decision, and it is what makes the anti-hallucination problem tractable: a language model that is only allowed to describe grounded evidence can be checked mechanically against that evidence; a model that decides cannot.

Component	May do	Must never do
Causal backbone (engineered graph)	Define causal structure: nodes, edges, delay windows, conditions — every edge traceable to a fault/event tree (PRA·PSA) or a configuration record.	Accept a learned or LLM-proposed edge without engineer confirmation and a versioned model change.
Data-informed layer	Re-weight existing edges from historian data; propose candidate edges for engineer review.	Define structure. Candidates are held out of the backbone until confirmed; rejected candidates stay visible as rejected.
Inference core (deterministic)	Generate candidate roots, forward-simulate each against the alarm set, score (coverage · timing fit · parsimony · prior), calibrate, rank.	Consume free-text input; produce prose. Its inputs are events and the graph; its output is a ranked, evidenced hypothesis set.
DSLM layer (Ollama, local)	Retrieve grounding evidence; compose the cited explanation of the ranked result; answer operator questions from the corpus; abstain when grounding is absent.	Generate, score, reorder or suppress hypotheses; write to any store; reference an entity that does not exist in the database; answer without citations.
Human (operator / engineer)	Confirm or reject the root cause. A confirmed cause closes into the incident record and the hash-chained audit log, and feeds model governance.	Be bypassed. No conclusion is committed without confirmation; the engine never closes an investigation on its own.

Decision-authority table — the contract every later section implements.

2 · Architecture at one look



Left of the database the pipeline is numeric and deterministic; right of it, linguistic and grounded. The database is the hinge: the inference core writes its ranked hypotheses and evidence links into it, and the DSLM is permitted to read only what is in it — the causal model, the run results, the document corpus and the registry. Anything not in the database does not exist as far as the language model is concerned, which is precisely the property the anti-hallucination controls in §8 enforce.

In Phase 0 the left half of this pipeline exists in simplified, in-browser form (Part A); the full pipeline is the production target (Part B). The decision-authority contract is identical in both.

3 · Part A — the Phase-0 mechanism, as built

Everything in this section runs in the current single-file console; the figures quoted are computed by the working code. The Root Cause Graph page carries three views — Causal Graph, Alarm Flood, Root-Cause Ranking — over three hand-authored incident reconstructions.

3.1 · The three-layer causal model

Causal structure is engineered, not discovered from telemetry. Inferring causal edges from observational time-series is unreliable under exactly the conditions a reactor presents — tight control-loop feedback, confounders and sensor lag routinely manufacture spurious edges. The graph therefore rests on an auditable backbone, with data used only to weight and propose, never to define:

Layer	Source	Role
1 · Engineered backbone	Plant fault / event trees (PRA·PSA), component topology, traceability records	Defines the causal and provenance edges. Every edge is traceable to an analysed failure path or a configuration record.
2 · Live runtime overlay	Active alarms and anomaly flags	Lights nodes by severity and animates propagation along the known edges. Adds state, not structure.
3 · Data-informed layer	Learned weights / candidate edges (ML)	Adjusts edge confidence and proposes candidate edges for engineer confirmation. Hints only — held out of the backbone until confirmed.

Four edge styles carry the meaning on screen: solid = engineered backbone; dashed violet = data-informed weight; dashed orange = artefact edge (an electrical or measurement disturbance corrupting a reading without a physical change); struck grey = a candidate edge proposed and then rejected, or an expected corroboration that is absent.

3.2 · The node test — forward propagation

Clicking any node tests it as the root cause: the engine walks the directed graph forward from that node along active edges, collects every downstream node whose alarm is in the current set, and reports “explains x / N alarms” together with the per-edge propagation delays. This is the elementary operation everything else builds on — the alarm-flood collapse runs it from the true origin; the ranking runs it for every candidate.

3.3 · Alarm-flood collapse

In incident EVT-2026-0418, fourteen alarms arrive in just over four minutes. The true origin — feedwater valve FV-104, actuator latency — is a single low-severity alarm buried in the middle of the flood, while high-severity downstream alarms dominate attention. Grouping each alarm by the propagation path it sits on collapses the fourteen raw alarms to one originating fault plus five cascade groups — a 93% noise suppression in the demonstration scenario. This is the ISA-18.2 alarm-management payoff: the operator is oriented, not drowning.

3.4 · Ranking — causal origin vs. alarm count

The ranking view scores each candidate two ways and lets the user flip between them. Alarm-count ranking sorts by direct alarms — it puts the loudest node on top (here the pump bearing), a downstream symptom. Causal-origin ranking sorts by a confidence derived from the edge weights and the explanatory reach of each candidate — it promotes the quiet upstream valve that explains the bulk of the cascade:

Hypothesis	Role	Direct alarms	Explains	Edge provenance	Causal confidence
FV-104 actuator latency	ORIGIN	1	12 / 14	fault-tree	**66%**
Pump 2A bearing degradation	PROXIMATE	2	4 / 14	fault-tree	20%
RCP-1B bearing wear	PARALLEL	2	3 / 14	fault-tree	7%
4kV bus voltage transient	CONTRIBUTING	1	3 / 14	data-informed	5%
Flow sensor FT-7 fault	RULED OUT	0	0 / 14	rejected	2%

Phase-0 confidences are illustrative, derived from edge weights — not calibrated probabilities and not identified from plant data. Part B replaces them with the scored, calibrated quantity of §4.

3.5 · Three incidents, three reasoning classes

Incident	Scenario	Reasoning class it demonstrates
EVT-2026-0418	Feedwater valve cascade — FV-104 latency → SG-1 level → pump overspeed → bearing → loop oscillation → trip	Upstream-origin inference: the quiet single-alarm origin outranks the loud downstream symptom; convergence of a parallel contributor (RCP-1B) is represented, not ignored.
EVT-2026-0419	Switchgear operation couples EMI into several instrument channels at once	Artefact vs. physical excursion: artefact edges model a reading corrupted without a physical change; the decisive evidence is absent corroboration from independent instruments on the same loop.
EVT-2026-0420	Non-conforming rod / QA escape reaches the core	Provenance reasoning: the causal path runs through traceability records rather than process physics — the backbone includes configuration edges, so a quality escape is a first-class root cause.

4 · Part B — how the production engine reaches a decision

The production inference core generalises the Phase-0 node test into a full diagnostic cycle. It corresponds to stages R1–R4 of the RCA roadmap; this section specifies the mechanism those stages build. The cycle below runs incrementally — it updates per arriving alarm rather than recomputing from scratch — and every step writes its intermediate result to the grounding database, so the final ranking is reproducible from logged inputs.

Step	Operation	Mechanism
1 · Condition	Clean the alarm stream	Debounce and chattering suppression, stale/shelved handling, dedup, first-out logic, priority per EEMUA 191 / ISA-18.2. Diagnostic quality cannot exceed alarm quality — nonsense in, nonsense out.
2 · Bind	Map signals to the model	Each conditioned event is bound tag→node via the binding layer, carrying timestamp, severity and a data-quality flag. Unbindable events are logged, never silently dropped.
3 · Generate	Candidate roots	Backward traversal from every alarmed node collects the set of upstream nodes that could, per the backbone, have initiated the pattern — the hypothesis set.
4 · Simulate	Forward propagation per candidate	Each candidate is propagated forward (the Phase-0 node test, generalised): which alarms it explains, and whether each observed arrival falls inside the edge's modelled delay window.
5 · Score	Four evidence dimensions	Coverage — fraction of the alarm set explained. Timing fit — arrival order and delay-window consistency; a cause must precede its effects. Parsimony — fewer assumed simultaneous faults score higher. Prior — component health and degradation state from the registry.
6 · Calibrate	Scores → confidence	A probabilistic layer (Bayesian / evidential) turns raw scores into calibrated confidence — stated confidence is required to match observed accuracy on the validation corpus, and is robust to missing, late and spurious alarms and to coincident faults.
7 · Rank & evidence	The decision	The ranked hypothesis set is written to the database with an evidence link per (hypothesis, alarm, edge) triple — the machine-readable "why". This record, not prose, is the engine's conclusion.
8 · Explain & confirm	Human-facing layer	The DSLM composes the cited explanation (§7); the operator or investigating engineer confirms or rejects. A confirmed cause closes into the incident record and the hash-chained audit log, and feeds governed model maintenance.

WHY TIMING, NOT JUST TOPOLOGY

Timing is the differentiator. Topology alone says a path is possible; the delay windows say whether this alarm pattern is consistent in time with that path. A cause must precede its effects within the modelled windows — timing turns a plausible graph path into a defensible diagnosis, and it is what separates true causes from coincidences.

5 · The grounding database — MS SQL Server

One relational database is the single source of truth for everything both halves of the pipeline are allowed to know: the governed causal model, the conditioned event history, every diagnosis run with its evidence, the document corpus with embeddings, and the full provenance of every DSLM exchange. The platform choice is Microsoft SQL Server (2025 or later), exercising the EF Core swap-in clause of Project Roadmap v2 decision D3; the historian role is served by partitioned, columnstore-indexed event tables, and vector retrieval by the native `VECTOR` type and distance functions (with a sidecar ANN index as the documented fallback on earlier versions).

5.1 · Schema — four groups

Causal model (schema `rc`) — the governed graph

Table	Key columns	Purpose
rc.Node	NodeId PK · UnitId · Kind (component | failure-mode | symptom | record) · RegistryRef	Graph nodes; components map 1:1 to Component Registry entries.
rc.Edge	EdgeId PK · FromNodeId · ToNodeId · Layer (fta | ml | art) · Status (confirmed | candidate | rejected) · Weight · DelayMinSec · DelayMaxSec · Condition · ProvenanceRef · ModelVersionId	Causal edges with delay windows and activation conditions. Layer and Status encode the three-layer model; ProvenanceRef points at the fault-tree gate, FMEA row or traceability record the edge derives from.
rc.ModelVersion	ModelVersionId PK · CreatedBy · ApprovedBy · ApprovedAt · ChangeNote · CoverageReport	Every structural change is a new approved version — the human-governed change control of the hybrid model. The engine always runs against exactly one version.

Runtime & diagnosis (schema `rc`) — what happened and what the engine concluded

Table	Key columns	Purpose
rc.AlarmEvent	EventId PK · Ts (UTC, ms) · TagId · NodeId · Severity · State · Quality · ConditioningFlags	The conditioned event stream (partitioned by time, columnstore). Ordering integrity is a hard requirement for temporal reasoning.
rc.DiagnosisRun	RunId PK · UnitId · WindowFrom/To · ModelVersionId · EngineVersion · StartedAt · TriggerEventId	One row per diagnostic cycle — pins the exact model and engine version, making the run replayable.
rc.Hypothesis	HypothesisId PK · RunId · NodeId · Coverage · TimingFit · Parsimony · Prior · Confidence · Rank · Tag (ORIGIN | PROXIMATE | PARALLEL | CONTRIBUTING | RULED-OUT)	The ranked output of §4 — the decision, as data.
rc.EvidenceLink	HypothesisId · EventId · EdgeId · InWindow bit · LagSec	One row per (hypothesis, alarm, edge): which alarm this hypothesis explains, via which edge, and whether the timing fit held. The machine-readable “why”.
rc.Confirmation	RunId · HypothesisId · ConfirmedBy · ConfirmedAt · Disposition · IncidentRef · AuditSealRef	The human decision, linked into the incident record and the hash-chained audit log.

Knowledge corpus (schema `kb`) — what the DSLM may know

Table	Key columns	Purpose
kb.Document	DocumentId PK · SourceType (internal | external) · Class (procedure | manual | event-report | regulatory | standard) · Title · Origin · Revision · Sha256 · ApprovedBy · ImportedAt	Every document in the corpus, hashed and approved at import (§6). Nothing enters at runtime.
kb.DocChunk	ChunkId PK · DocumentId · Section · PageFrom/To · ChunkText · Embedding VECTOR(1024) · TokenCount	Retrieval units with embeddings (native VECTOR; ANN sidecar fallback). Chunking is structure-aware — sections survive intact.
kb.IngestLog	DocumentId · Pipeline · EmbeddingModelTag · Outcome · Notes	Full provenance of every import, including which embedding model produced the vectors.

DSLM provenance (schema `ai`) — every exchange, reconstructible

Table	Key columns	Purpose
`ai.Exchange`	`ExchangeId PK · RunId NULL · Question · RetrievedChunkIds JSON · ContextHash · AnswerText · Citations JSON · Grounded bit · AbstainReason NULL · ModelTag · Temperature · Seed · LatencyMs · AuditSealRef`	One row per question/answer, including abstentions. ContextHash + ModelTag + Seed make the answer reproducible; AuditSealRef chains it into compliance.
`ai.RetrievalLog`	`ExchangeId · ChunkId · Score · Stage (vector | lexical | structured | rerank)`	Per-chunk retrieval scores at each stage — the grounding gate's audit trail.
`ai.EvalRun`	`EvalRunId PK · CorpusVersion · ModelTag · FaithfulnessScore · CitationPrecision · RefusalRecall · PassedGate bit`	Results of the evaluation harness (§8, H9) — the record behind the roadmap-v2 M4 gate.

Naming and types are indicative; the binding artefact in production is the migration set, version-controlled with the services.

Two access roles enforce the decision contract at the database boundary: the inference core holds read-write on `rc.*`; the DSLM service holds a read-only login covering all four schemas and nothing else. The language model is incapable of writing to the system that grounds it — by permission, not by convention.

6 · Knowledge sources & ingestion governance

The DSLM answers only from a curated, versioned corpus. Two source classes feed it, through one controlled door:

Class	Sources	Examples in this design
Internal — plant & platform	Operating procedures and technical specifications; vendor manuals; P&ID and traceability/configuration records; historical event reports; Component Registry snapshots; historian extracts; the platform's own documentation set.	The three documented incidents EVT-2026-0418 / -0419 / -0420 with their causal reconstructions; the Reactor Physics Reference; the Functional Guide; registry entries such as FV-104.
External — curated industry knowledge	Regulatory and operating-experience literature imported as documents, reviewed and approved like any other: IAEA safety reports and IRS operating experience; US NRC licensee event reports and generic communications; EPRI guidance; WANO significant operating-experience reports; EEMUA 191 / ISA-18.2 alarm-management practice.	An NRC LER describing a feedwater-valve oscillation event becomes retrievable context when the engine ranks a valve hypothesis — clearly cited as external operating experience, never blended invisibly with plant fact.

Ingestion governance — the air gap is preserved

No runtime network access exists. The Ollama host and the database live in the isolated environment; the model cannot browse, fetch or call anything. External knowledge enters only through an offline, reviewed import: a document is proposed, reviewed for relevance and licensing, approved by a named person, hashed (SHA-256), chunked and embedded by a pinned local embedding model, and recorded in `kb.IngestLog`. Corpus changes are therefore versioned events with provenance — and every corpus version triggers a re-run of the evaluation harness (H9) before it is served. The corpus can be rebuilt bit-identically from the import set.

> **DESIGN RULE**
> The corpus boundary is the knowledge boundary. If a fact is not in an approved document, the registry or the diagnosis record, the system treats it as unknown — and says so. "The model probably knows this from pre-training" is never an accepted source.

7 · The retrieval pipeline (RAG)

When the operator asks a question — or the engine requests an explanation for a finished run — the DSLM service executes a fixed retrieval sequence. Generation is the last step, and the cheapest one to refuse:

Stage	Mechanism
1 · Query framing	The question is paired with structured context: the active DiagnosisRun, the selected unit, and any entity IDs it mentions (validated against the registry before use).
2 · Structured lookup	Deterministic SQL first: the hypothesis row, its evidence links, the registry entries and incident records for every referenced entity. Facts that can be fetched exactly are never left to similarity search.
3 · Hybrid retrieval	Vector search over kb.DocChunk embeddings and lexical full-text search, in parallel — lexical catches exact tags and clause numbers that embeddings blur.
4 · Re-rank & assemble	Candidates are re-ranked against the query; the top chunks are assembled into a context block in which every chunk carries its ChunkId and document citation, within a fixed token budget.
5 · Grounding gate	Before any generation: if no chunk clears the relevance floor, or the evidence count is below minimum, the service returns the abstention template (§8, H2). The model is never asked to "do its best".
6 · Generation	The pinned Ollama model generates under the closed-world prompt contract (H1) at temperature ≈0 with a fixed seed — structured output, every claim citing a ChunkId or a database row.
7 · Verification	Post-generation checks (H3–H5) validate citations, entities and schema before anything reaches the operator. Failures regenerate once, then abstain.

8 · Anti-hallucination design — defence in depth

A language model asked about something outside its evidence will, by default, produce fluent fiction. In this domain that is not a quality problem but a safety-culture problem, so the design does not rely on the model behaving — it relies on ten independent controls, most of them mechanical checks outside the model entirely. The governing rule: when grounding is absent, the correct output is a refusal, and a refusal is a first-class, logged, audited answer — not a failure state.

#	Control	Mechanism	Catches
H1	**Closed-world prompt contract**	The system prompt restricts the model to the supplied context block and structured rows; it must cite a source token for every claim and must output the abstention form when the context does not contain the answer.	Pre-training “knowledge” leaking in as if it were plant fact.
H2	**Grounding gate (pre-generation)**	A relevance floor on retrieval scores plus a minimum-evidence count, enforced by the service before the model is invoked. Below threshold → abstention template, logged with AbstainReason.	Questions the corpus simply cannot answer — the largest hallucination class is removed before generation.
H3	**Entity validation**	Every component, tag, incident or document ID appearing in the draft answer is checked against the registry and corpus tables. Unknown ID → reject.	Invented equipment, invented incidents, plausible-looking but nonexistent references.
H4	**Citation verification**	Every cited ChunkId / row reference must be a member of the actually-retrieved set for this exchange (ai.RetrievalLog). Claims without a surviving citation are stripped; if the answer loses its substance, the exchange abstains.	Fabricated citations; claims smuggled in beside genuine ones.
H5	**Structured output schema**	The model answers in a fixed JSON form — claims[], each with source refs; confidence is copied from rc.Hypothesis, never generated. Schema violations reject the draft.	Free-prose drift; the model inventing or inflating confidence numbers.
H6	**Determinism & pinning**	Temperature ≈0, fixed seed, pinned model tag and pinned corpus version; ContextHash recorded. Same question on the same corpus ⇒ the same answer.	Irreproducible answers — the audit requirement of roadmap-v2 P8.7 applied to language output.
H7	**No write path**	The DSLM service holds a read-only database login and has no API surface toward the engine, the model store or the audit chain.	Any route by which generated text could contaminate the decision system.
H8	**Abstention as first-class output**	The UI renders “insufficient grounding in the approved corpus” as a normal, styled answer with the retrieval summary attached — and the operator can see what was searched.	The product-design failure where refusal looks broken, pressuring the system toward guessing.
H9	**Evaluation harness & gate**	A golden Q&A set scored for faithfulness and citation precision, plus trap questions whose only correct answer is a refusal (refusal recall). Re-run on every corpus or model change; serving is gated on passing — the roadmap-v2 M4 criterion.	Regression, drift, and quiet degradation after updates.
H10	**Audit chaining**	Every exchange — question, retrieved set, answer or abstention, verification outcomes — is sealed into the hash-chained compliance log via AuditSealRef.	Untraceable advice; enables after-the-fact review of every word shown to an operator.

WHY TEN LAYERS, NOT ONE GOOD PROMPT

H1 is a request; H2–H7 are enforcement. The prompt asks the model to behave — the gate, the validators, the schema and the read-only login make misbehaviour unobservable at the operator’s screen. Defence in depth means no single layer, including the model itself, is trusted.

9 · Worked example — one question through the whole mechanism

Setting: the EVT-2026-0418 alarm flood. The inference core has completed a run: FV-104 ranked ORIGIN (coverage 12/14, timing fit consistent, parsimony best, confidence 66%). The operator asks:

> *"FV-104 is throwing a cluster of alarms — what is going on?"*

Stage	What happens
Structured lookup	SQL fetches the run's top hypothesis row, its 12 evidence links, the FV-104 registry entry (actuator latency rising) and the EVT-2026-0418 incident record.
Hybrid retrieval	Vector + lexical search returns: the event-report chunk describing the feedwater-valve cascade pattern; the procedure section on valve-actuator response checks; an external NRC LER on a comparable feedwater oscillation (cited as external operating experience).
Grounding gate	Scores clear the floor with five evidence items → generation proceeds.
Generation + verification	The model composes the answer below; H3 confirms FV-104 and EVT-2026-0418 exist; H4 confirms all citations are in the retrieved set; H5 validates the schema and copies the 66% figure from rc.Hypothesis.
Rendered answer	"The FV-104 alarm cluster matches the feedwater-valve cascade recorded as EVT-2026-0418: a sticking control valve drives a feedwater-flow oscillation that trips level and flow alarms together. The registry shows FV-104 actuator latency rising, and the engine's current run ranks it as the originating fault (confidence 66%, explaining 12 of 14 alarms). The alarms are correlated, not independent — confirm actuator response before resetting the downstream alarms. Advisory only — operator confirms." — followed by its citation list: Component Registry (FV-104) · rc.Hypothesis (run) · EVT-2026-0418 · procedure §x · NRC LER (external).
Audit	The exchange — question, retrieved ChunkIds, answer, citations, checks — is sealed into the compliance chain.

9.1 · The counter-example — the mechanism refusing

The operator then asks about a valve that exists in the plant vocabulary but has no records in this system:

> *"What is the maintenance history of valve FV-220?"*

Entity validation finds no FV-220 in the registry; retrieval over the corpus returns nothing above the relevance floor. The grounding gate fires before generation, and the operator sees a styled, honest answer:

> **ABSTENTION — A FIRST-CLASS ANSWER**
>
> "Insufficient grounding: no component 'FV-220' exists in the Component Registry, and no approved document in the corpus mentions it. I cannot answer this from the available sources. Searched: registry (0 matches), corpus (0 chunks above threshold). If this component exists, its records have not been imported — see ingestion governance."

A baseline LLM would have invented a plausible maintenance history here. Under this mechanism the invention is structurally impossible to display: there was nothing to cite, so there was nothing to say — and the refusal itself is logged, sealed and reviewable.

10 · Limitations & non-claims

- The causal structure is engineered from fault/event trees and configuration records — the platform does not perform causal discovery from telemetry, and learned or LLM-proposed edges never enter the backbone unconfirmed.
- Phase-0 confidences are illustrative edge-weight derivations; production confidences are calibrated against a curated event corpus — and are still triage quantities, not a PRA result.
- The DSLM explains decisions and answers from the corpus; it does not diagnose. Removing the DSLM entirely leaves the diagnostic capability intact — that asymmetry is the design.
- Diagnostic quality is capped by alarm quality and proven only by validation (RCA roadmap R1, R8); this document specifies mechanism, not demonstrated field performance.
- Everything here is an advisory and investigative aid: no control authority, no interaction with certified, hardware-isolated reactor protection — in any mode, in any phase.

Exists & runs today (Phase 0)	Specified here, built later (production)
Three-layer graph, node test, 14→1 alarm-flood collapse, dual ranking, three worked incidents — in-browser, deterministic, every displayed number computed.	Conditioned Kafka stream, temporal-probabilistic inference core, MS SQL Server grounding database, Ollama DSLM with the H1–H10 controls, evaluation harness and audit chaining.